

目录

第 1 节	实验背景与数据集	1
1	实验背景	1
2	数据集	1
3	数据预处理	1
1	归一化处理	1
2	标签处理	1
3	输入展平	1
4	添加偏置项	1
第 2 节	实验目的	2
第 3 节	实验方法	2
1	网络结构设计	2
2	网络结构示意图	2
3	各模块说明	3
1	Matmul（矩阵乘法层）	3
2	ReLU 激活函数	3
3	Softmax	3
4	交叉熵损失函数	3
4	优化方法	3
1	L2 正则化	3
2	动量梯度下降	3
3	学习率衰减	4
第 4 节	实验结果	4
1	训练参数	4
2	训练过程现象	4
3	最终结果	4
4	训练过程可视化	4
第 5 节	实验分析与总结	5
1	网络性能分析	5
2	参数影响	5
3	优化效果	5
4	实验总结	5
第 6 节	附录	5
1	主要代码模块	5
2	核心代码	6

第 1 节 实验背景与数据集

1 实验背景

本实验构建了一个三层全连接神经网络。输入处理：将 MNIST 数据集中的 28×28 灰度图像展平为 784 维向量，并在输入层添加偏置项（全 1 列），得到 785 维输入。

网络结构：第一层全连接层将输入从 785 维映射到 256 维，后接 ReLU 激活函数；第二层全连接层将 256 维映射到 128 维，后接 ReLU 激活函数；第三层全连接层将 128 维映射到 10 维，后接 Softmax 函数输出各类别概率。

权重初始化：采用 He 初始化方法，各层权重按 $\text{scale} = \sqrt{2.0/\text{input_dim}}$ 设置随机初始值。

模块实现：将矩阵乘法、ReLU 激活、Softmax 输出、交叉熵损失分别封装为独立类，每个类均实现 forward 前向传播和 backward 反向传播方法，便于梯度计算与参数更新。

2 数据集

本实验使用 **MNIST 手写数字数据集**：

- 训练集：60000 张图片
- 测试集：10000 张图片
- 图片大小： 28×28 （灰度图）
- 分类类别：10 类（数字 0-9）

3 数据预处理

1 归一化处理

$$x = x/255.0$$

2 标签处理

标签转为 One-hot 编码。

3 输入展平

$$28 \times 28 \rightarrow 784$$

4 添加偏置项

输入层添加偏置项（+1）。

第 2 节 实验目的

- 理解前向传播与反向传播的基本原理
- 掌握全连接神经网络（MLP）的结构设计
- 学会使用 NumPy 手动实现神经网络各模块

第 3 节 实验方法

1 网络结构设计

本实验构建了一个三层全连接神经网络。

网络结构：第一层全连接层将输入从 785 维映射到 256 维，后接 ReLU 激活函数；第二层全连接层将 256 维映射到 128 维，后接 ReLU 激活函数；第三层全连接层将 128 维映射到 10 维，后接 Softmax 函数输出各类别概率。

权重初始化：采用 He 初始化方法，各层权重按 $\text{scale} = \sqrt{2.0/\text{input_dim}}$ 设置随机初始值。

模块实现：将矩阵乘法、ReLU 激活、Softmax 输出、交叉熵损失分别封装为独立类，每个类均实现 forward 前向传播和 backward 反向传播方法，便于梯度计算与参数更新。

2 网络结构示意图

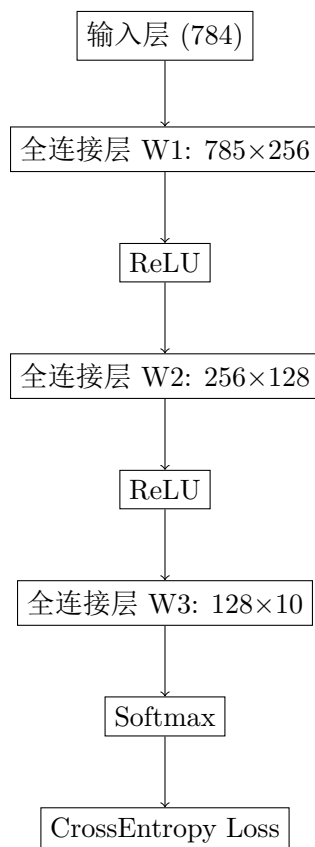


图 3.1 网络结构示意图

3 各模块说明

1 Matmul（矩阵乘法层）

前向传播：

$$y = xW$$

反向传播：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} W^T$$
$$\frac{\partial L}{\partial W} = x^T \frac{\partial L}{\partial y}$$

2 ReLU 激活函数

$$f(x) = \max(0, x)$$

梯度：

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

3 Softmax

$$S_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

4 交叉熵损失函数

$$L = - \sum y \log(\hat{y})$$

4 优化方法

1 L2 正则化

$$L = L + \lambda \|W\|^2$$

2 动量梯度下降

$$v = \mu v + \eta \nabla W$$

$$W = W - v$$

3 学习率衰减

每 10 个 epoch:

$$lr = lr \times 0.95$$

第 4 节 实验结果

1 训练参数

参数	取值
batch_size	64
epochs	80
初始学习率	0.001
momentum	0.9

表 4.1 训练参数设置

2 训练过程现象

- 初期 loss 快速下降, accuracy 迅速提升至 95%+
- 后期收敛变慢, 学习率衰减提高稳定性

3 最终结果

- 训练集准确率: 约 99%+
- 测试集准确率: 约 98%+

4 训练过程可视化

训练过程中的 Loss 曲线、Accuracy 曲线和 Learning Rate 曲线如图所示。

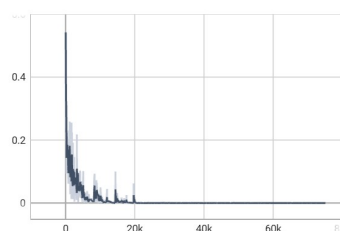


图 4.1 Loss 曲线

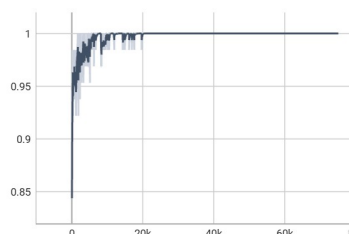


图 4.2 Accuracy 曲线

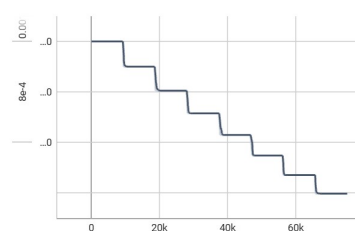


图 4.3 Learning Rate 曲线

第 5 节 实验分析与总结

1 网络性能分析

- 三层网络已能较好拟合 MNIST
- ReLU 提高非线性表达能力
- Softmax + 交叉熵稳定训练

2 参数影响

参数	影响
层数增加	表达能力增强但易过拟合
神经元数量	过少欠拟合，过多计算量大
学习率	过大震荡，过小收敛慢
batch_size	影响稳定性与速度

表 5.1 参数影响分析

3 优化效果

- 动量：加快收敛
- L2 正则化：防止过拟合
- 学习率衰减：提升后期稳定性

4 实验总结

本实验通过纯 NumPy 实现了一个完整的神经网络训练流程，深入理解了：

- 神经网络前向传播与反向传播机制
- 各种层（Matmul、ReLU、Softmax）的作用
- 损失函数与优化算法原理
- 深度学习训练流程

该实验强化了对深度学习底层实现的理解，为后续使用框架（如 PyTorch）奠定了基础。

第 6 节 附录

1 主要代码模块

- 数据加载：mnist_dataset()

- 网络结构: myModel
- 前向传播: forward()
- 反向传播: backward()
- 训练函数: train_one_step()

2 核心代码

Code Listing 1 代码实现

```
import os
import numpy as np
from tensorflow.keras import datasets
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
from torch.utils.tensorboard import SummaryWriter
#训练集保存在C:\Users\Admin\.keras\datasets\

def mnist_dataset():
    (x,y),(x_test,y_test) = datasets.mnist.load_data()
    #归一化
    x = x/255.0
    x_test = x_test/255.0
    return (x,y),(x_test,y_test)

#矩阵乘法类
class Matmul():
    def __init__(self):
        self.mem = {}
    def forward(self,x,W):
        h = np.matmul(x,W)
        self.mem = {"x":x,"W":W}
        return h
#矩阵乘法运算的反向传播
    def backward(self,grad_y):
        x = self.mem["x"]
        W = self.mem["W"]
        #.T转置
        grad_x = np.matmul(grad_y,W.T)
        grad_W = (np.matmul(x.T,grad_y))
        return grad_x,grad_W

#定义ReLU类
class Relu():
    def __init__(self):
        self.mem = {}
    def forward(self,x):
        self.mem["x"] = x
```

```
#ReLU激活函数:大于0返回原值, 小于等于0返回0
return np.where(x>0,x,np.zeros_like(x))
def backward(self, grad_y):
    x = self.mem["x"]
    return (x>0).astype(np.float32)*grad_y

#Softmax:先进行e指数求解, 再进行归一化
# 定义 softmax 类
class Softmax():
    def __init__(self):
        # 构造函数
        self.mem = {}
        self.epsilon = 1e-12 # 防止分母出现 0

    # softmax 函数的前向传播
    def forward(self, x):
        # x_shape(N, c)
        x_exp = np.exp(x)
        denominator = np.sum(x_exp, axis=1, keepdims=True)
        # softmax 的分母
        out = x_exp / (denominator + self.epsilon)
        # softmax 的输出, 对应式 (4)
        self.mem["out"] = out
        self.mem["x_exp"] = x_exp
        # 将 out 和 x_exp 保存在字典 mem 中, 用于反向传播中的梯度计算
        return out

    # softmax 函数的反向传播
    def backward(self, grad_y):
        # grad_y: same shape as x
        s = self.mem["out"] # shape (N, c)
        sisj = np.matmul(np.expand_dims(s, axis=2),
                           np.expand_dims(s, axis=1))
        g_y_exp = np.expand_dims(grad_y, axis=1) # shape: (N, 1, c)
        tmp = np.matmul(g_y_exp, sisj) # shape: (N, 1, c)
        tmp = np.squeeze(tmp, axis=1) # shape: (N, c)
        # squeeze()为expand_dims()的逆向操作
        softmax_grad = -tmp + grad_y*s # 对应式(9)
        return softmax_grad

    # 返回softmax函数的梯度

# 定义交叉熵类
class Cross_entropy():
    # 构造函数
    def __init__(self):
        self.epsilon = 1e-12
        self.mem = {}
```



```
# 交叉熵函数的前向传播
def forward(self, x, labels):
    log_prob = np.log(x + self.epsilon) # 对输出的概率取对数
    out = np.mean(np.sum(-log_prob*labels, axis=1))
    # 对应式(1), axis=1表示沿水平方向求和
    self.mem["x"] = x
    return out

# 返回交叉熵, 表示loss

# 交叉熵函数的反向传播
def backward(self, labels):
    x = self.mem["x"]
    return -1/(x + self.epsilon)*labels # 对应式(3)

# 建立模型
class myModel():
    def __init__(self):
        scale1 = np.sqrt(2.0/(28*28 + 1))
        scale2 = np.sqrt(2.0/256)
        scale3 = np.sqrt(2.0/128)
        self.W1 = np.random.normal(0,scale1,size = [28*28+1, 256])
        self.W2 = np.random.normal(0,scale2,size = [256, 128])
        self.W3 = np.random.normal(0,scale3,size=[128,10])

        self.v_W1 = np.zeros_like(self.W1)
        self.v_W2 = np.zeros_like(self.W2)
        self.v_W3 = np.zeros_like(self.W3)
        self.momentum = 0.9

        self.mul_h1 = Matmul()
        self.relu1 = Relu()
        self.mul_h2 = Matmul()
        self.relu2 = Relu()
        self.mul_h3 = Matmul()
        self.softmax = Softmax()
        self.cross_en = Cross_entropy()

#生成类的实例

##模型前向传播
def forward(self, x, labels):
    x = x.reshape(-1, 28*28) #将x铺平
    bias = np.ones(shape=[x.shape[0], 1])
    x = np.concatenate([x, bias], axis=1)

#第一层
```

```
self.h1 = self.mul_h1.forward(x, self.W1)
self.h1_relu = self.relu1.forward(self.h1)

# 第二层
self.h2 = self.mul_h2.forward(self.h1_relu, self.W2)
self.h2_relu = self.relu2.forward(self.h2)

# 第三层
self.h3 = self.mul_h3.forward(self.h2_relu, self.W3)
self.h3_soft = self.softmax.forward(self.h3)

# 计算loss
self.loss = self.cross_en.forward(self.h3_soft, labels)

# 模型反向传播
def backward(self, labels):
    # 交叉熵函数对输入求梯度
    self.loss_grad = self.cross_en.backward(labels)

    # softmax层梯度
    self.h3_soft_grad = self.softmax.backward(self.loss_grad)

    # 第三层梯度
    self.h3_grad, self.W3_grad = \
        self.mul_h3.backward(self.h3_soft_grad)

    # 第二层relu层梯度
    self.h2_relu_grad = self.relu2.backward(self.h3_grad)

    # 第二层矩阵乘法梯度
    self.h2_grad, self.W2_grad = self.mul_h2.backward(self.h2_relu_grad)

    # 第一层ReLU梯度
    self.h1_relu_grad = self.relu1.backward(self.h2_grad)

    # 第一层矩阵乘法梯度
    self.h1_grad, self.W1_grad = self.mul_h1.backward(self.h1_relu_grad)
model = myModel()

# 计算准确率
def compute_accuracy(prob, labels):
    predictions = np.argmax(prob, axis=1)
    # 返回输出的概率每一行最大值所在的列数
    truth = np.argmax(labels, axis=1)
    # 返回标签每一行最大值所在的列数
    return np.mean(predictions==truth)

# 迭代一个 epoch
```

```
def train_one_step(model, x, y, lr=1e-3, l2_lambda=0.0001):
    model.forward(x, y)
    model.backward(y)

    # 添加L2正则化
    model.W1_grad += l2_lambda * model.W1
    model.W2_grad += l2_lambda * model.W2
    model.W3_grad += l2_lambda * model.W3

    # 使用动量更新
    model.v_W1 = model.momentum * model.v_W1 + lr * model.W1_grad
    model.v_W2 = model.momentum * model.v_W2 + lr * model.W2_grad
    model.v_W3 = model.momentum * model.v_W3 + lr * model.W3_grad

    model.W1 -= model.v_W1
    model.W2 -= model.v_W2
    model.W3 -= model.v_W3

    loss = model.loss
    accuracy = compute_accuracy(model.h3_soft, y) # 改为h3_soft
    return loss, accuracy

# 计算测试集上的 loss 和准确率
def test(model, x, y):
    model.forward(x, y)
    loss = model.loss
    accuracy = compute_accuracy(model.h3_soft, y) # 改为h3_soft
    return loss, accuracy

# 实际训练
train_data, test_data = mnist_dataset()
train_label = np.zeros(shape=[train_data[0].shape[0],10])

# 生成0矩阵
test_label = np.zeros(shape=[test_data[0].shape[0],10])
train_label[np.arange(train_data[0].shape[0]),np.array(train_data[1])] = 1

# 将标签转换成one-hot矩阵
test_label[np.arange(test_data[0].shape[0]),np.array(test_data[1])] = 1

# 添加batch训练
batch_size = 64
epochs = 80
initial_lr = 1e-3
global_step = 0

writer = SummaryWriter("../logs_mnist_numpy")
```

```
for epoch in range(epochs):
    # 打乱数据
    indices = np.arange(train_data[0].shape[0])
    np.random.shuffle(indices)
    train_x_shuffled = train_data[0][indices]
    train_label_shuffled = train_label[indices]

    total_loss = 0
    total_acc = 0
    num_batches = 0

    # Batch训练

    for i in range(0, train_data[0].shape[0], batch_size):
        batch_x = train_x_shuffled[i:i + batch_size]
        batch_y = train_label_shuffled[i:i + batch_size]

        # 学习率衰减 (每10个epoch衰减一次)
        lr = initial_lr * (0.95 ** (epoch // 10))

        loss, accuracy = train_one_step(model, batch_x, batch_y, lr=lr)
        total_loss += loss
        total_acc += accuracy
        num_batches += 1

    # 记录到 TensorBoard
    writer.add_scalar('Train/Loss', float(loss), global_step)
    writer.add_scalar('Train/Accuracy', float(accuracy), global_step)
    writer.add_scalar('Train/Learning_Rate', float(lr), global_step)

    if global_step % 100 == 0:
        sample_images = batch_x[:4].reshape(-1, 1, 28, 28)
        writer.add_images("Input_Images", sample_images, global_step)
        global_step += 1

    if i % (batch_size * 100) == 0:
        print(f'Epoch {epoch}, Batch {i // batch_size}: loss={loss:.4f},
acc={accuracy:.4f}')

    avg_loss = total_loss / num_batches
    avg_acc = total_acc / num_batches
    print(f'Epoch {epoch} finished: avg_loss={avg_loss:.4f}, avg_acc={avg_acc:.4f}')

    # 每5个epoch测试一次
    if epoch % 5 == 0:
        test_loss, test_acc = test(model, test_data[0], test_label)
        print(f'Test after epoch {epoch}: loss={test_loss:.4f}, acc={test_acc:.4f}')
```

```
# 最终测试
writer.close()
test_loss, test_acc = test(model, test_data[0], test_label)
print(f'Final test loss: {test_loss:.4f}, accuracy: {test_acc:.4f}')
```